

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting (BL 3)

Assessment Tool Weightage: 40%

QUESTIONS: 15

Total Marks: 25

Annexure A Constraint Based Problem Solving

Code of Conduct:

I V.Madhulika(192312620) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

v. madhulika

Signature of the student

Q1. Patient health records CSV: design a constraint-based data cleaning pipeline ensuring (i) no missing values, (ii) numerical values within medically valid ranges, (iii) standardized categorical fields.

Answer:

Step 1 – Load and Profile

- Load the CSV with pandas (`pd.read_csv`) and use `df.info()`, `df.isnull().sum()` to profile missing values, dtypes, and cardinality of categorical columns.

Step 2 – Handle Missing Values (Constraint i)

- Use `df.fillna()` with column-wise strategies: median imputation for skewed vitals (e.g., blood pressure), mean for near-normal features, and mode for categorical fields.
- For columns where >50% values are missing, drop the column with `df.drop(columns=...)` instead of imputing, to avoid injecting noise.
- Re-verify with `df.isnull().sum().sum() == 0` as the exit condition.

Step 3 – Enforce Valid Ranges (Constraint ii)

- Define a dictionary of medically valid ranges (e.g., heart rate 40–200, SpO2 60–100) and use boolean-mask filtering: `df = df[(df['hr'] >= 40) & (df['hr'] <= 200)]` or clip out-of-range values with `df['hr'].clip(40, 200)` when values are likely sensor drift rather than true outliers.
- Use NumPy's `np.where()` for vectorized conditional correction across large columns instead of row-wise loops.

Step 4 – Standardize Categorical Fields (Constraint iii)

- Normalize case/spacing with `str.strip().str.lower()`, then map synonyms (e.g., 'M', 'male', 'Male' → 'Male') using a mapping dict with `df['col'].replace(mapping)`.
- Convert to pandas 'category' dtype to enforce a fixed set of allowed labels going forward.

Indexing, Filtering & Aggregation

- Use `.loc[]` for label-based conditional edits so only rows matching the constraint are modified, preserving the rest of the data intact.
- Aggregate with `groupby('patient_id').agg()` to check for duplicate/conflicting records per patient before finalizing, and log any rows that fail a constraint to a separate 'rejected_records.csv' rather than silently dropping them, so data integrity is preserved and auditable.

Q2. Design a memory-efficient solution to perform math operations, random sampling, and statistical analysis on a massive NumPy dataset without exceeding memory limits.

Answer:

Efficient Array Creation

- Use the smallest sufficient dtype (`np.float32` instead of `float64`, `np.int16` instead of `int64`) via `dtype=` at creation — this alone can cut memory by 50–75%.
- Use `np.memmap()` to map the array to disk rather than loading it fully into RAM, or process the file in chunks with `np.load(..., mmap_mode='r')`.

In-place Operations

- Use in-place operators (`arr += 1`, `np.add(arr, 1, out=arr)`) instead of `arr = arr + 1`, which avoids allocating a new array copy each time.

- Use views/slicing (`arr[:2]`) instead of `np.copy()` wherever a copy isn't strictly required.

Random Sampling & Statistics at Scale

- Use `np.random.Generator` (`np.random.default_rng()`) with a fixed chunk size, sampling indices rather than materializing the whole permutation.
- Compute statistics with streaming/online algorithms (Welford's method) or chunked reduction: process the array in batches, accumulate sum and sum-of-squares, then derive mean/variance at the end instead of loading everything at once.

Optimizing Indexing & Aggregation

- Prefer boolean masks and `np.where` over fancy indexing with large index lists (fancy indexing always copies).
- Use axis-wise aggregation (`np.mean(arr, axis=0)`) which NumPy vectorizes internally rather than Python loops.

Debugging Memory/Performance Issues

- Use `memory_profiler` (`@profile` decorator) and `tracemalloc` to locate the exact line causing a spike.
- Check `arr.nbytes` and `arr.flags` to confirm whether an operation created an unwanted copy (OWNDATA flag).
- If overflow persists, switch to chunked processing with generators or Dask arrays, which mimic the NumPy API but operate lazily out-of-core.

Q3. Combine two large Pandas DataFrames (transactions + demographics) with constraints: no duplicate customer IDs, consistent indexing after merge, and preservation of all valid records.

Answer:

Combining Strategy

- Use `pd.merge(transactions, demographics, on='customer_id', how='left')` so every valid transaction record is preserved (constraint iii) even if a demographic match is missing.
- Reset the index immediately after merging with `.reset_index(drop=True)` to guarantee a clean, consistent `0...n-1` index (constraint ii).

Preventing Duplicate Customer IDs

- Before merging, deduplicate the demographics side with `drop_duplicates(subset='customer_id', keep='last')` since it should be one row per customer; verify with `demographics['customer_id'].is_unique`.
- After merging, assert `transactions.shape[0] == merged.shape[0]` to confirm the merge didn't fan-out rows (a sign of duplicate keys on the right side).

Validating with Filtering & Grouping

- Filter for orphan rows using `merged[merged['demo_col'].isna()]` to see transactions with no demographic match, and log them separately.
- Use `groupby('customer_id').size()` on the merged result — any count greater than the original transaction count per customer signals unwanted duplication.

Sorting & Indexing

- Sort with `sort_values(['customer_id', 'transaction_date'])` then `set_index('customer_id')` for fast customer-level lookups downstream.

Debugging Mismatches

- Use `indicator=True` in `pd.merge` to add a `'_merge'` column ('left_only', 'right_only', 'both') that immediately shows which records failed to match, then investigate those keys for typos, whitespace, or type mismatches (str vs int IDs).

Q4. Compute mean, median, and variance on a dataset with constraints on computational efficiency and numerical accuracy, using aggregation/statistical functions, filtering and indexing.

Answer:

Efficient, Accurate Computation

- Use NumPy/Pandas built-in vectorized functions (`df.mean()`, `df.median()`, `df.var()`) rather than manual Python loops — they're implemented in C and numerically stable.
- For very large sums, use `np.float64` accumulation even if the source data is `float32`, to avoid precision loss from repeated rounding (Kahan-summation-style accuracy is already used internally by NumPy's pairwise summation).

Avoiding Redundant Computation

- Compute all needed statistics in a single pass with `df.describe()` or `df.agg(['mean', 'median', 'var'])` instead of calling each separately, since this reuses intermediate sums.
- Cache the result if the same statistic is needed multiple times downstream, rather than recomputing.

Filtering & Indexing to Exclude Irrelevant Data

- Use boolean masks (`df[df['value'].notna() & (df['value']!=0)]`) to exclude nulls/placeholder zeros before aggregating, since including them silently biases the mean.
- Use `.loc[]` with a condition to restrict computation to a relevant subset (e.g., only `'valid' == True` rows) instead of computing over the whole table and discarding results.

Debugging Incorrect Statistical Outputs

- Cross-check with a manual formula on a small known sample to validate the pipeline logic.
- Check for hidden NaNs (`df.isna().sum()`) and dtype issues (object dtype numbers stored as strings won't aggregate correctly) — a common silent cause of wrong results.
- Compare `ddof=0` vs `ddof=1` settings in variance calculations, since population vs sample variance is a frequent source of 'incorrect' answers that are actually just a convention mismatch.

Q5. Design a complete data analysis pipeline reading multiple files, processing via Pandas, and generating visualizations, with constraints on I/O efficiency, correct grouping/sorting, filtering, and meaningful plots.

Answer:

Stage 1 – Loading (Efficient File I/O)

- Use `glob` to collect all file paths, then `pd.concat([pd.read_csv(f) for f in files])` for CSVs, or `pd.read_parquet` for large binary files (faster and smaller than CSV).
- Specify `dtype` and `usecols` in `read_csv` to avoid loading unnecessary columns and to prevent pandas from inferring inefficient types.

Stage 2 – Processing / Filtering

- Apply user-defined boolean conditions via `df.query()` or boolean masks to keep only relevant rows before heavier processing, reducing memory and compute downstream.

Stage 3 – Aggregation (Grouping & Sorting)

- Use `df.groupby(['category']).agg({'sales':'sum','qty':'mean'})` for correct multi-metric aggregation, then `.sort_values('sales', ascending=False)` to rank groups meaningfully.

Stage 4 – Plotting

- Use matplotlib/seaborn on the aggregated (already-small) DataFrame — never plot raw millions of rows directly. Choose the chart type by intent (bar for category comparison, line for trend over time).

Debugging Corrupted Files / Inconsistent Formats

- Wrap each file read in `try/except`, logging the filename on failure instead of stopping the whole pipeline.
- Standardize column names and encodings (`encoding='utf-8', errors='ignore'`) right after each load, before concatenation, so schema drift between files doesn't break the merge.

Q6. Financial transactions with duplicates, inconsistent timestamps, invalid entries: design a cleaning pipeline that removes duplicates without losing valid data, standardizes time formats, and filters invalid transactions.

Answer:

Removing Duplicates Safely

- Use `df.drop_duplicates(subset=['txn_id'], keep='first')` rather than a full-row duplicate check, since two genuinely different valid transactions could otherwise share incidental field values.
- Before dropping, group by the suspected duplicate key with `groupby('txn_id').size()` and manually inspect any group `>1` to confirm they truly are duplicates, not legitimate repeats (e.g., recurring payments).

Standardizing Timestamps

- Convert all date columns with `pd.to_datetime(df['ts'], errors='coerce', utc=True)` which normalizes multiple formats and time zones into one consistent dtype; invalid strings become `NaT` and can then be isolated.

Filtering Invalid Transactions

- Define validity rules (`amount > 0`, currency in allowed set, ts not null) and filter with a combined boolean mask: `df = df[(df['amount']>0) & df['ts'].notna()]`.

Indexing, Grouping & Sorting for Integrity

- Set a `DateTimeIndex` (`set_index('ts')`) after cleaning to enable time-based slicing and resampling.
- Group by `account_id` to check post-cleaning balances/counts against expected totals as a sanity check, and sort by timestamp to confirm chronological consistency for downstream fraud/pattern analysis.

Q7. Real-time sensor streaming data under strict latency constraints: design filtering, normalization, and anomaly detection using NumPy arrays with minimal delay and memory.

Answer:

Filtering & Normalization

- Use a fixed-size circular/ring buffer (a pre-allocated NumPy array with a rolling write index) instead of appending to a growing list, avoiding repeated reallocation.
- Normalize incoming values in-place using vectorized operations: `buffer[:] = (buffer - running_mean) / running_std`, updated incrementally rather than recomputed from scratch each tick.

Anomaly Detection

- Use a simple, fast statistical rule (z-score beyond ± 3 , or exponentially weighted moving average/variance) computed via vectorized NumPy operations, which is $O(1)$ per new reading when done incrementally.

Minimizing Latency & Memory

- Use slicing (`buffer[-window:]`) instead of copying subsets, and rely on broadcasting/vectorization instead of per-element Python loops, since loops dominate latency in streaming code.
- Pre-allocate all arrays at startup; avoid dynamic resizing inside the hot loop.

Debugging Delayed Processing / Incorrect Outputs

- Profile the per-tick loop with `time.perf_counter()` around each stage to isolate the bottleneck (I/O read vs computation vs anomaly check).
- Log buffer statistics periodically to catch drift between the running mean/variance estimate and the true windowed value, which signals an accumulation bug in the incremental update formula.

Q8. Merge multiple large product-information datasets from different sources with constraints: resolving conflicting attribute values, maintaining consistent schema, avoiding data loss.

Answer:

Schema Alignment

- Standardize column names across sources first (a rename mapping per source) so `pd.concat/merge` operations align on the same keys — schema drift is the most common cause of silent data loss.

Merging & Conflict Resolution

- Merge with `pd.merge(..., how='outer', suffixes=('_src1', '_src2'))` so no rows from either source are dropped (constraint iii), producing duplicate attribute columns for conflicting fields.
- Resolve conflicts with an explicit priority rule, e.g., `df['price'] = df['price_src1'].combine_first(df['price_src2'])` to prefer the more trusted source while falling back to the other when missing.

Validation with Grouping & Filtering

- Use `groupby('product_id').nunique()` on each attribute to detect where sources still disagree after resolution, and filter those rows out for manual review rather than silently averaging incompatible categorical values.
- Confirm row-count conservation: `len(merged)` should equal the union of unique `product_ids` across sources, verified with set operations.

Q9. Transform a mixed-data-type dataset with millions of records into a structured ML-ready format: efficient type conversion, handling missing/categorical data, consistent indexing.

Answer:

Efficient Type Conversion

- Use `pd.to_numeric(df['col'], errors='coerce', downcast='integer')` and `astype('category')` for low-cardinality text columns — category dtype dramatically reduces memory versus object dtype.
- Convert in bulk with `df.convert_dtypes()` as a first pass, then fine-tune specific columns.

Handling Missing & Categorical Data

- Impute numeric NaNs with median (robust to outliers) and categorical NaNs with a new explicit 'Unknown' category rather than dropping rows, to avoid losing scale on a millions-row dataset.
- Encode categoricals with `pd.get_dummies()` for low-cardinality fields or an ordinal/frequency encoding for high-cardinality fields (dummies would explode column count).

Consistent Indexing

- Reset to a clean `RangeIndex` after all transformations (`reset_index(drop=True)`) so downstream ML train/test splitting works predictably.

Efficiency at Scale

- Process the conversion in chunks with `pd.read_csv(..., chunksize=100000)` and write each transformed chunk to a Parquet file incrementally, rather than holding the full transformed dataset in memory at once.

Q10. Compute rolling statistics (moving average, rolling variance) on a time-series dataset under high-performance and accuracy constraints.

Answer:

Implementation

- Set a proper `DateTimeIndex` (`df.set_index('timestamp')`) then use the built-in, C-optimized rolling API: `df['ma'] = df['value'].rolling(window=10).mean()` and `df['value'].rolling(window=10).var()`.
- These pandas rolling methods use an efficient incremental algorithm internally, avoiding the $O(n \cdot \text{window})$ cost of a naive recomputation loop.

Efficient Indexing & Minimal Redundancy

- Ensure the index is sorted (`sort_index()`) before rolling, since an unsorted `DateTimeIndex` silently produces wrong windows.
- Use `min_periods` to control behavior at the start of the series instead of manually padding, and chain rolling operations (e.g., `.rolling().mean().rolling().std()`) sparingly to avoid recomputation passes.

Debugging Misalignment / Bottlenecks

- Visually spot-check a small manual window calculation against the `rolling()` output for the first few rows.

- If timestamps have irregular gaps, resample to a fixed frequency first (`resample('1min').mean()`) before rolling — otherwise `'window=10'` means 10 rows, not 10 minutes, which is a very common alignment bug.
- For performance bottlenecks on huge series, switch to `df.rolling(window).agg()` with the numba engine (`engine='numba'`) for a JIT-compiled speed-up.

Q11. Healthcare dataset with mixed CSV and JSON formats: design a preprocessing pipeline that unifies schemas, handles nested JSON efficiently, and ensures no inconsistency after transformation.

Answer:

Unifying Formats into One Schema

- Load CSV with `pd.read_csv()` and JSON with `pd.json_normalize()` (which flattens nested structures into flat columns), then align both to a single agreed column-name schema via a rename mapping before concatenation.

Handling Nested JSON Efficiently

- Use `json_normalize(data, record_path=..., meta=...)` to expand nested lists (e.g., multiple visits per patient) into separate rows while keeping parent-level fields (`patient_id`) attached — this avoids manual recursive parsing.

Ensuring No Inconsistency

- After flattening, cast shared fields to identical dtypes across both sources (e.g., `patient_id` as string in both) since JSON often preserves numeric types that CSV loses as strings.
- Concatenate with `pd.concat([csv_df, json_df], ignore_index=True)` only after both share identical column names and order.

Indexing, Normalization, Aggregation

- Reindex on `patient_id` after merging, and use `groupby('patient_id').agg()` to collapse multiple nested records (e.g., multiple lab readings) into per-patient summary statistics.

Debugging Schema Mismatches

- Diff the column sets with `set(csv_df.columns) ^ set(json_df.columns)` to immediately surface fields present in one source but not the other, and resolve before concatenation rather than after.

Q12. Financial analytics system computing real-time risk metrics (returns, volatility, correlations) from streaming stock data under strict latency constraints using NumPy.

Answer:

Vectorized Computation

- Compute returns with a single vectorized operation: `returns = np.diff(prices) / prices[:-1]` instead of a Python loop, since NumPy's C-level diff is orders of magnitude faster.
- Compute rolling volatility as the standard deviation over a sliding NumPy view (using stride tricks or a small fixed window array) and correlations with `np.corrcoef()` on the returns matrix across assets — a single call handles the whole cross-asset matrix at once.

Minimizing Memory & Maximizing Speed

- Use broadcasting (e.g., normalizing all asset columns against a market index column in one operation) rather than looping per asset.
- Maintain fixed-size pre-allocated arrays for the streaming window and slide them with `np.roll()` or index-wrapping rather than growing lists.

Slicing & Aggregation

- Slice only the latest window (`prices[-window:]`) for each incremental update instead of recomputing from the full history.

Debugging Incorrect/Delayed Outputs

- Validate correlation/volatility figures against a known static historical calculation on the same data slice.
- Time each computation stage; if correlation calculation dominates latency, reduce update frequency for that metric relative to faster metrics like returns, which is a standard mitigation for latency-critical systems.

Q13. Clean a large dataset of social media posts (text, timestamps, user metadata): remove duplicate/spam entries, normalize text, and ensure consistent timestamp formatting.

Answer:

Removing Duplicates & Spam

- Use `drop_duplicates(subset=['user_id','text'])` for exact duplicates, and a similarity/heuristic filter (e.g., repeated identical text from the same user within seconds, or excessive links/hashtags) via string methods and groupby-based frequency counts to flag likely spam.

Normalizing Text

- Use vectorized string operations: `df['text'].str.lower().str.strip().str.replace(r'http\S+', '', regex=True)` to remove URLs, and strip emojis/special characters with a regex pass — all via pandas' vectorized `.str` accessor rather than per-row Python loops.

Consistent Timestamp Formatting

- Standardize with `pd.to_datetime(df['ts'], errors='coerce', utc=True)`, coercing unparseable formats to NaT for isolation and review.

Filtering, Grouping, String Processing

- Group by `user_id` and time-bucket to detect burst-posting spam patterns, and filter out posts failing basic quality checks (empty after cleaning, non-target language).

Debugging Noisy/Inconsistent Data

- Sample and manually inspect a random subset after each cleaning stage (`df.sample(20)`) to catch cleaning rules that are too aggressive or too lax, since automated metrics alone can miss subtle noise.

Q14. Feature extraction from a high-dimensional dataset under computational efficiency and scalability constraints: feature selection, normalization, dimensionality reduction.

Answer:

Feature Selection

- Compute a correlation matrix (`df.corr()`) and drop one of each highly correlated pair ($|r| > 0.9$) to remove redundancy cheaply before any heavier modeling.
- Use variance thresholding (drop near-constant columns) as a fast first-pass filter, since it's $O(n)$ and removes clearly uninformative features immediately.

Normalization

- Apply vectorized Z-score or Min-Max scaling with NumPy/ sklearn's `StandardScaler`, fit only on a representative sample if the dataset is too large to fit in memory, then transform in chunks.

Dimensionality Reduction

- Apply PCA (`sklearn.decomposition.PCA`) on the standardized data, selecting the number of components that retain $\sim 95\%$ explained variance — this both compresses the feature space and speeds up downstream training.

Indexing, Vectorization, Aggregation

- Use column-wise vectorized operations throughout (never row-wise apply loops) and process in batches with incremental PCA (`IncrementalPCA`) for datasets too large for a single fit.

Debugging Redundant/Irrelevant Features

- Inspect PCA component loadings to see which original features dominate each component — features contributing to no significant component are candidates for removal, and re-validate model performance before/after removal to confirm no information loss.

Q15. Integrate and analyze data from multiple IoT devices with varying sampling rates and missing timestamps: align time-series correctly, handle missing intervals, maintain efficiency.

Answer:

Aligning Time-Series Data

- Set each device stream's `DateTimeIndex` and resample all devices to a common fixed frequency (`resample('1min').mean()`) so streams with different native sampling rates become comparable on the same time grid.

Handling Missing Intervals

- Use `reindex()` against a complete `date_range` to expose missing timestamps explicitly, then interpolate with `.interpolate(method='time')` for short gaps (which respects uneven original spacing) rather than a naive forward-fill that would distort trends.
- For long gaps, leave values as `NaN` (rather than interpolating across large spans) to avoid fabricating misleading data — this preserves distortion-free integrity per the constraint.

Maintaining Computational Efficiency

- Merge/align using `pd.concat(axis=1)` on already-resampled, common-index DataFrames rather than row-by-row timestamp matching, which is vectorized and scales far better across many devices.

Debugging Misaligned/Inconsistent Time-Series

- Plot a short time window across devices to visually confirm alignment after resampling.

- Check for timezone inconsistencies (tz-naive vs tz-aware timestamps) across devices, which is a very common silent cause of apparent misalignment, and standardize all streams to UTC before merging.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand